Modul 4 Soal 1: “Brankas Nicole”

Deskripsi

Suatu hari yang damai di ibu kota New Eridu, Pantheon menerima sebuah teka-teki dari Nicole untuk membuka brankas rahasia yang katanya berisi banyak denny (mata uang di New Eridu).

Namun, ada satu masalah...
Brankas ini dikunci menggunakan tiga angka kunci, dan angka-angka tersebut tersembunyi dalam sebuah pohon biner misterius.

Nicole memberikan daftar angka yang digunakan untuk membangun pohon biner berdasarkan level order traversal. Dari pohon tersebut, ketiga angka kunci dapat ditemukan menggunakan inorder traversal.

Pantheon sudah berusaha tapi dia mengalami kesulitan untuk menyelesaikan kode program ini. Oleh karena itu, ia meminta Fairy AI paling cerdas di New Eridu (yaitu kamu!) untuk membantunya menyelesaikan kode program tersebut.

Ehn-na-ehn-nuh? (Bisakah kamu membantu Pantheon dan Nicole membuka brankas ini?)

Input Format

- Baris pertama berisi sebuah bilangan bulat N, yaitu jumlah node dalam pohon.
- Baris kedua berisi N bilangan bulat yang dipisahkan oleh spasi, merepresentasikan urutan elemen berdasarkan level order traversal.
- Baris ketiga berisi tiga bilangan bulat, masing-masing menyatakan posisi elemen yang ingin diambil dari hasil inorder traversal (menggunakan indeks 1-based).

Constraints

- $1 \leq N \leq 100$
- $1 \leq$ Nilai setiap node $\leq 10^5$
- $1 \leq K \leq N$

Output Format

Cetak tiga angka dari hasil inorder traversal sesuai dengan posisi yang diberikan pada input baris ketiga. Tampilkan angka-angka tersebut dalam satu baris, dipisahkan dengan spasi, berurutan sesuai urutan posisi pada input.

Contoh Input:

```
7
10 20 30 40 50 60 70
3 4 1
```

Contoh Output:

```
50 10 40
```

Kode Soal

```cpp
#include <iostream>
using namespace std;

// Class Node
class Node {
public:
    int data;
    Node *left, *right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

// Class Binary Tree
class BST {
public:
    Node *root;
    BST() : root(nullptr) {}

    void buildFromLevelOrder(int arr[], int n) {
        // add your code here
    }

    void inorder(Node *node, int &count, int k, int &kthValue) {
        // add your code here
    }

    int findKthInorder(int k) {
        int count = 0, kthValue = -1;
        inorder(root, count, k, kthValue);
        return kthValue;
    }

};

int main() {
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int k[3];
    for (int i = 0; i < 3; i++) {
        cin >> k[i];
    }

    BST tree;

    // add your code here

    return 0;
}
```

Modul 4 Soal 2: “Quantum Pixel Art”

Deskripsi

Di sebuah laboratorium kecil yang penuh dengan gadget aneh dan tumpukan buku tebal tentang fisika kuantum, Professor Okabe Rintarou mendapat sebuah pesan misterius melalui komputer lamanya.

Pesan itu bukan sembarang pesan, tapi sebuah **deretan pixel berwarna** yang tampak seperti kode rahasia dari masa depan. **Awalnya, semua pixel berwarna putih bersih**, tapi seiring waktu, beberapa pixel mulai berubah warna sesuai instruksi yang dikirim oleh sebuah entitas tak dikenal.

Okabe tahu ini bukan sekadar kode biasa. Ada dua jenis perintah yang harus dia jalankan:

- Update satu pixel: Ubah warna pixel di posisi i menjadi warna baru c.
- Query satu pixel: Tentukan warna pixel di posisi i.

Namun, masalahnya adalah, Okabe tidak bisa menyimpan semua pixel satu per satu secara manual — terlalu banyak data yang harus diolah. Dia membutuhkan algoritma canggih untuk melacak perubahan tanpa kehilangan satu pun informasi.

Okabe meminta bantuanmu untuk membuat program yang dapat memproses semua perubahan pixel dan akhirnya menampilkan hasil akhir sebelum sinyal itu hilang selamanya.

Input Format

- Baris pertama berisi dua bilangan bulat, N (jumlah pixel) dan T (jumlah operasi).
- T baris berikutnya mendeskripsikan operasi:
  Jika operasinya adalah update, diberikan dalam format: 1 i C (ubah pixel di posisi i menjadi warna C).
  Jika operasinya adalah query, diberikan dalam format: 2 i (query warna pixel di posisi i).

Constraints

- 1 <= N <= 10^5
- 1 <= T <= 10^5
- 1 <= i <= N
- C : huruf kapital paling depan warna (white W, red R)

Output Format

- Untuk setiap operasi query, cetak warna pixel di posisi i.
- Setelah semua operasi, cetak keadaan akhir semua pixel dalam satu baris.

Sample Input

```
5 6
1 2 R
1 4 B
```

```
2 2
2 3
1 3 Y
2 3
```

Sample Output

```
R
W
Y
WRYBW
```

Kode Soal

```
#include <iostream>
using namespace std;

class AVLTree {
private:
    struct Node {
        int index;
        char color;
        int height;
        Node* left;
        Node* right;

        Node(int idx, char col) {
            index = idx;
            color = col;
            height = 1;
            left = right = nullptr;
        }
    };

    Node* root;

    int max(int a, int b) {
        // add your code here
    }

    int getHeight(Node* node) {
        // add your code here
    }

    int getBalance(Node* node) {
        // add your code here
    }

    Node* rotateRight(Node* y) {
        // add your code here
    }

    Node* rotateLeft(Node* x) {
        // add your code here
    }

    Node* insert(Node* node, int idx, char col) {
        // add your code here
    }

    char search(Node* node, int idx) {
        // add your code here
    }

    void printPixels(Node* node, int& cur, int N) {
        // add your code here
    }

public:
    AVLTree() {
        root = nullptr;
    }

        //update pixel
    void update(int idx, char col) {
        // add your code here
    }

        //query pixel
    char query(int idx) {
```

```
        // add your code here
    }

    void printFinal(int N) {
        // add your code here
    }
};

int main() {
    int N, T;
    cin >> N >> T;

    AVLTree tree;

    // add your code here (e.g., input loop)

    return 0;
}
```

Modul 4 Soal 3: “Day 1 as Storage Intern”

Deskripsi

🎉 Selamat!

Kamu diterima sebagai Storage Intern di Supermarket Together! Tugasmu di hari pertama ini adalah untuk mencari barang di Storage Room yang kapasitasnya **paling cocok** dengan kebutuhan rak di toko.

🧊 Deskripsi Jobdesk Baru Kamu

1. Setiap kali barang baru datang, kamu akan menambahkannya ke Storage Room. Setiap barang punya kapasitas tertentu (misal: 10, 20, dst).
2. Ketika suatu rak di supermarket butuh diisi ulang, manajer akan memberitahu kapasitas yang dibutuhkan (misal: 9, 15, dst).
3. Kamu harus mencari barang di Storage Room **yang kapasitasnya paling pas untuk kebutuhan rak**, yaitu:
    - Pilih barang dengan kapasitas **terkecil yang lebih besar dari kebutuhan rak** (misal, butuh 9, ambil 10; butuh 15, ambil 20).
    - **Jika ada barang yang kapasitasnya pas banget dengan kebutuhan rak, JANGAN diambil** (karena tidak profesional untuk mengosongkan stok, harus selalu ada sisa).
    - Jika tidak ada barang yang kapasitasnya lebih dari kebutuhan rak, print `-1`.
4. Setiap barang yang sudah diambil akan dikeluarkan dari Storage Room.

Jenis Perintah:

- `i X` : Masukkan barang dengan kapasitas X ke Storage Room.
- `j Y` : Cari dan berikan barang yang sesuai dengan kapasitas Y, sesuai ketentuan yang ada pada jobdesk.

🤔 Saran dari Supermarket Manager

Biar urusan stok dan cari produk nggak bikin pusing, coba pakai AVL Tree, pencarian lebih cepat dan storage room tetap rapi tanpa harus bolak-balik cek satu-satu.

Input Format

- Baris pertama berisi sebuah bilangan bulat Q, yaitu jumlah perintah yang akan diberikan.
- Diikuti Q baris, yang berisi dua jenis perintah:
  - `i X`
  - `j Y`

Constraints

- TL: 1s
- 1 ≤ Q ≤ 10^8
- 1 ≤ X ≤ 10^7
- 0 ≤ Y ≤ 10^7

Output Format

Untuk setiap perintah `j Y`, cetak satu baris berisi kapasitas barang yang diambil dari Storage Room, atau `-1` jika tidak ada barang yang memenuhi kriteria.

Sample Input

```
8
i 30
i 20
j 15
j 15
j 15
i 15
j 15
j 14
```

Sample Output

```
20
30
-1
-1
15
```

Explanation

1. **i 30**
   Storage Room: [30]
   (Barang dengan kapasitas 30 dimasukkan.)

2. **i 20**

Storage Room: [20, 30]
(Barang dengan kapasitas 20 dimasukkan, storage sekarang berisi 20 dan 30.)

3. **j 15**
- Butuh barang dengan kapasitas lebih dari 15 (pilihan: 20, 30).
- Pilih yang terkecil, yaitu 20.
Output: **20**
Storage Room: [30]

4. **j 15**
- Sekarang hanya ada 30 di storage.
- 30 > 15, jadi ambil 30.
Output: **30**
Storage Room: []

5. **j 15**
- Storage Room kosong, tidak ada barang yang bisa diambil.
Output: **-1**

6. **i 15**
Storage Room: [15]
(Barang dengan kapasitas 15 dimasukkan.)

7. **j 15**
- Ada barang dengan kapasitas **15** di storage.
- Tapi **tidak boleh diambil** karena harus selalu ada sisa (tidak boleh persis sama dengan permintaan).
- Tidak ada barang lain yang lebih dari 15.
Output: **-1**

8. **j 14**
- Ada barang dengan kapasitas lebih dari 14, yaitu 15.
- 15 > 14, jadi diambil 15.
Output: **15**
Storage Room: []

Kode Soal

```
#include <iostream>
#include <vector>
using namespace std;

class AVLTree
{
private:
    struct AVLNode
    {
```

```
    // Data members
};
AVLNode *_root;
unsigned int _size;

int _max(int a, int b) const
{
    // Helper function to get the maximum of two integers
}

int _getHeight(AVLNode* node) const
{
    // Helper function to get the height of a node
}

int _getBalanceFactor(AVLNode* node) const
{
    // Helper function to get the balance factor of a node
}

AVLNode* _rightRotate(AVLNode* pivotNode)
{
    // Right rotation function
}

AVLNode* _leftCaseRotate(AVLNode* node)
{
    // Left case rotation function
}

AVLNode* _leftRotate(AVLNode* pivotNode)
{
    // Left rotation function
}

AVLNode* _rightCaseRotate(AVLNode* node)
{
    // Right case rotation function
}

AVLNode* _leftRightCaseRotate(AVLNode* node)
{
    // Left-right case rotation function
}

AVLNode* _rightLeftCaseRotate(AVLNode* node)
{
    // Right-left case rotation function
}

AVLNode* _createNode(int value)
{
    // Function to create a new AVL node
}

AVLNode* _search(AVLNode *currRoot, int value) const
{
```

```
        // Function to search for a value in the AVL tree
    }

    AVLNode* _insert_recursive(AVLNode* node, int value)
    {
       // Function to insert a value into the AVL tree
    }

    AVLNode* _findMinNode(AVLNode *node) const
    {
        // Function to find the minimum node in the AVL tree
    }

    AVLNode* _remove_recursive(AVLNode* node, int value)
    {
       // Function to remove a value from the AVL tree
    }

    void _destroyRecursive(AVLNode* node)
    {
        // Function to destroy the AVL tree recursively
    }

    void _inorderRecursive(AVLNode* node, vector<int>& result) const
    {
        // Function to perform an inorder traversal of the AVL tree
    }

public:
    AVLTree() : _root(nullptr), _size(0u) {}

    ~AVLTree()
    {
        // Destructor to clean up the AVL tree
    }

    bool find(int value) const
    {
        // Function to find a value in the AVL tree
    }

    void insert(int value)
    {
        // Function to insert a value into the AVL tree
    }

    void remove(int value)
    {
        // Function to remove a value from the AVL tree
    }

    // Add more functions as needed
};

int main()
{
    // Add your code here
```

```
}
```

Modul 4 Soal 4: “Order of the Deep”

Deskripsi

Kamu datang ke rinascita untuk menyelidiki tentang order of the deep, akan tetapi karena Rinascita merupakan pulau terpencil dikelilingi lautan berbahaya maka disana masih menerapkan sistem feodal. sistem jabatan disini dapat digambarkan dengan binary tree. maka agar tidak salah dalam beretika buatlah program simple yang bisa membantu dirimu menyelesaikan penyelidikan dan menguak rahasia order of the deep dengan mudah.

Input Format

- Baris pertama berisi bilangan asli N, yaitu jumlah operasi yang akan dilakukan
- ‘N’ Baris berikutnya berisi salah satu dari 3 jenis operasi :
    - A <int> <string> -> Menambahkan node dengan value <int> bernama <string> ke dalam tree
    - B <string> <string> -> 2 <string> bertemu
        - string yang rendah posisinya akan membungkuk sebanyak X kali kepada string yang lebih tinggi posisinya pada tree dengan format
            - “<string> membungkuk X kali ke <string>”
        - apabila posisi mereka sama maka mereka akan bersalaman dengan format
            - “<string> dan <string> bersalaman”
    - C -> Node kecil akan melakukan coup d'etat dan tree akan melakukan rotasi clockwise

Constraints

- $1 \leq N \leq 10^3$
- $1 \leq \text{<int>} \leq 10^5$
- <string> hanya terdiri dari huruf kecil dan hanya 1 kata tanpa spasi

Output format

- Untuk setiap perintah B <string1> <string2>, output akan dikeluarkan sesuai keadaan
    - <string1> berada pada posisi lebih tinggi maka output :
        - “<string2> membungkuk X kali ke <string1>”
    - <string2> berada pada posisi lebih tinggi maka output :
        - “<string1> membungkuk X kali ke <string2>”
    - <string1> dan <string2> berada pada posisi sama :
        - “<string1> dan <string2> bersalaman” (yang disebut lebih dahulu pada input muncul lebih dahulu)

Contoh input 0

```
11
A 5 fenrico
A 3 phoebe
A 7 fisalia
A 2 montelli
B montelli fisalia
```

A 4 fools
A 6 maiden
C
A 8 bard
B fools phoebe
B bard maiden

Contoh output 0

montelli membungkuk 1 kali ke fisalia
fools membungkuk 2 kali ke phoebe
bard dan maiden bersalaman

Kode Soal

```
#include <iostream>
#include <string>
using namespace std;

class Node {
public:
	int key;
	string name;
	Node* left;
	Node* right;

	Node(int k, const string& n) : key(k), name(n), left(nullptr),
right(nullptr) {}
};

class BinaryTree {
public:
	Node* root;

	BinaryTree() : root(nullptr) {}

	Node* insert(Node* node, int key, const string& name) {
	// Recursively insert a new node with the given key and name into
the correct position in the BST
	}

	void insert(int key, const string& name) {
	// Public insert method that starts insertion from the root
	}
```

```
      void rotate_clockwise() {
      // Perform a right rotation (clockwise) at the root node if
possible
      }

      int find_depth(Node* node, const string& name, int depth) {
      // Recursively search for a node by name and return its depth in
the tree
      }

      int find_depth(const string& name) {
      // Public method to find the depth of a node by name, starting
from the root
      }

      bool exists(Node* node, const string& name) {
      // Recursively check if a node with the given name exists in the
subtree
      }

      bool exists(const string& name) {
      // Public method to check if a node with the given name exists in
the tree
      }
};

int main() {
      // Read input and perform operations (insert, rotate, or compare
depths) based on commands
}
```

Modul 4 Soal 5: “Orb: Thaumazein”

Deskripsi

Kamu adalah murid dari Rafal, seorang astrolog abad ke-15. Bermula dari suatu kejadian yang tidak dapat diceritakan di sini, kamu mulai ragu dengan metode gurumu. Dia menyuruhmu untuk percaya, tetapi di saat yang bersamaan, dia melakukan sesuatu yang justru membuatmu semakin ragu.

Karena tekanan tersebut, kamu pun berusaha mencari jalanmu sendiri untuk menemukan keseimbangan antara keraguan dan kepercayaan.

Kamu menggambarkan perjalanan ini dalam bentuk sebuah AVL tree, yang melambangkan keseimbangan itu sendiri. Namun, dalam ketidakpastian, kamu harus menentukan apakah pohon ini lebih condong pada keraguan, kepercayaan, atau justru mencapai kebenaran.

Dapatkah kamu menemukan kebenaran dalam dunia yang penuh ketidakpastian ini?

"Wandering is how you find what's truth"

Input Format

- Baris pertama adalah bilangan bulat N, yaitu jumlah operasi yang akan dilakukan.
- `N` baris berikutnya berisi salah satu dari dua jenis operasi:
  - I X → Memasukkan bilangan X ke dalam AVL Tree.
  - S → Periksa keseimbangan dan cetak hasilnya:
    - "Keraguan" jika lebih condong ke kiri.
    - "Kebenaran" jika seimbang.
    - "Kepercayaan" jika lebih condong ke kanan.

Constraints

- $1 \leq N \leq 10^3$
- $1 \leq X \leq 10^5$

Output Format

Cetak satu string yang menunjukkan keadaan keseimbangan tree ("Keraguan", "Kebenaran", atau "Kepercayaan").

Contoh Input 0:

```
4
I 30
I 20
I 40
S
```

Contoh Output 0:

```
Kebenaran
```

Contoh Input 1:

```
7
I 30
I 20
I 40
I 10
I 25
I 50
S
```

Contoh Output 1:

```
Keraguan
```

Contoh Input 2:

```
6
I 30
I 20
I 40
I 50
I 60
S
```

Contoh Output 2:

```
Kepercayaan
```

Kode Soal

```cpp
#include <iostream>
using namespace std;

class Node {
public:
    int value;
    int height;
    Node* left;
    Node* right;

    Node(int val) {
        value = val;
        height = 1;
        left = right = nullptr;
    }
};

// Class AVL Tree
class AVLTree {
private:
    Node* root;

    int height(Node* node) {
        // add your code here
```

```
    }

    int getBalance(Node* node) {
        // add your code here
    }

    Node* rightRotate(Node* y) {
        // add your code here
    }

    Node* leftRotate(Node* x) {
        // add your code here
    }

    Node* insert(Node* node, int value) {
        // add your code here
    }

    void countNodes(Node* node, int& leftCount, int& rightCount) {
        // add your code here
    }

public:
    AVLTree() { root = nullptr; }

    void insert(int value) {
        root = insert(root, value);
    }

    void checkBalance() {
        if (!root) {
            cout << "Kebenaran" << endl;
            return;
        }
        int leftCount = 0, rightCount = 0;
        countNodes(root, leftCount, rightCount);
        if (leftCount > rightCount)
            cout << "Keraguan" << endl;
        else if (leftCount < rightCount)
            cout << "Kepercayaan" << endl;
        else
            cout << "Kebenaran" << endl;
    }
};

int main() {
    int N;
    cin >> N;

    AVLTree avl;

    // add your code here

    return 0;
}
```